week - 1(a)

```
import java.util.Scanner;

class QuadraticRoots {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double a, b, c, d, r1, r2;

        System.out.print("Enter a, b and c: ");
        a = sc.nextDouble();
        b = sc.nextDouble();
        c = sc.nextDouble();

        d = b * b - 4 * a * c;

        if (d > 0) {
            r1 = (-b + Math.sqrt(d)) / (2 * a);
            r2 = (-b - Math.sqrt(d)) / (2 * a);

            System.out.println("Roots are real and distinct");
            System.out.println("Root 1 = " + r1);
            System.out.println("Root 2 = " + r2);
        } else if (d == 0) {
            r1 = -b / (2 * a);

            System.out.println("Roots are real and equal");
            System.out.println("Root 1 = " + r1);
            System.out.println("Root 2 = " + r1);
        } else {
            double real = -b / (2 * a);
            double imaginary = Math.sqrt(-d) / (2 * a);

            System.out.println("Roots are complex");
            System.out.println("Root 1 = " + real + " + " + imaginary
+ "i");
            System.out.println("Root 2 = " + real + " - " + imaginary
+ "i");
        }

        sc.close();
    }
}
```

week - 1(b)

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
```

```java
class PrimeNumbers {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in));

        System.out.print("Enter the limit: ");
        int n = Integer.parseInt(br.readLine());

        System.out.println("Prime numbers up to " + n + " are:");

        for (int i = 2; i <= n; i++) {
            int count = 0;

            for (int j = 1; j <= i; j++) {
                if (i % j == 0) {
                    count++;
                }
            }

            if (count == 2) {
                System.out.print(i + " ");
            }
        }
    }
}
```

week - 2(a)

```java
import java.util.Scanner;

class Palindrome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        String rev = "";

        for (int i = str.length() - 1; i >= 0; i--) {
            rev = rev + str.charAt(i);
        }

        if (str.equalsIgnoreCase(rev)) {
            System.out.println("The string is a palindrome");
        } else {
            System.out.println("The string is not a palindrome");
        }

        sc.close();
```

```
    }
}
```

week - 2(b)

```
import java.util.Arrays;

class SortStrings {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Please enter strings as command-line
arguments.");
            return;
        }

        Arrays.sort(args);

        System.out.println("Sorted strings are:");

        for (String str : args) {
            System.out.println(str);
        }
    }
}
```

week - 3(a)

1.

```
class Overloading {
    void add(int a, int b) {
        System.out.println("Sum = " + (a + b));
    }

    void add(int a, int b, int c) {
        System.out.println("Sum = " + (a + b + c));
    }

    public static void main(String[] args) {
        Overloading obj = new Overloading();

        obj.add(10, 20);
        obj.add(10, 20, 30);
    }
}
```

2.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
```

```java
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Overriding {
    public static void main(String[] args) {
        Animal obj = new Dog();
        obj.sound();
    }
}
```

week - 3(b)

1.

```java
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class SingleInheritance {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

2.

```java
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

```
class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy weeps");
    }
}

class MultilevelInheritance {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.weep();
    }
}
3.

interface Printable {
    void print();
}

interface Showable {
    void show();
}

class Demo implements Printable, Showable {
    public void print() {
        System.out.println("Printing...");
    }

    public void show() {
        System.out.println("Showing...");
    }
}

class MultipleInheritance {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.print();
        d.show();
    }
}

week - 4(a)

package mypack;

public class Message {
    public void display() {
        System.out.println("Welcome to Java Packages");
```

```
    }
}

import mypack.Message;

class TestPackage {
    public static void main(String[] args) {
        Message obj = new Message();
        obj.display();
    }
}
javac -d . Message.java
javac TestPackage.java
java TestPackage

week - 4(b)

abstract class Shape {
    abstract void area();

    void display() {
        System.out.println("Calculating area");
    }
}

interface Printable {
    void print();
}

class Circle extends Shape implements Printable {
    double radius = 5;

    void area() {
        System.out.println("Area of Circle = " + (3.14 * radius *
radius));
    }

    public void print() {
        System.out.println("This is a Circle");
    }
}

class AbstractInterfaceDemo {
    public static void main(String[] args) {
        Circle c = new Circle();

        c.display();
        c.area();
        c.print();
```

```
    }
}

week - 5(a)

class ExceptionHandling {
    public static void main(String[] args) {
        int a = 10, b = 0;

        try {
            int c = a / b;
            System.out.println("Result = " + c);
        } catch (ArithmeticException e) {
            System.out.println("Exception: Division by zero");
        } finally {
            System.out.println("Finally block executed");
        }

        System.out.println("Program continues...");
    }
}

week - 5(b)

class AgeException extends Exception {
    AgeException(String message) {
        super(message);
    }
}

class UserDefinedException {
    public static void main(String[] args) {
        int age = 16;

        try {
            if (age < 18) {
                throw new AgeException("Age must be 18 or above");
            }

            System.out.println("Eligible to vote");
        } catch (AgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        System.out.println("Program continues...");
    }
}

week - 6

class Buffer {
```

```
    int item;
    boolean available = false;

    synchronized void produce(int value) {
        try {
            while (available) {
                wait();
            }

            item = value;
            System.out.println("Produced: " + item);
            available = true;
            notify();

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    synchronized void consume() {
        try {
            while (!available) {
                wait();
            }

            System.out.println("Consumed: " + item);
            available = false;
            notify();

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

class Producer extends Thread {
    Buffer b;

    Producer(Buffer b) {
        this.b = b;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            b.produce(i);
        }
    }
}

class Consumer extends Thread {
```

```
    Buffer b;

    Consumer(Buffer b) {
        this.b = b;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            b.consume();
        }
    }
}

class ProducerConsumer {
    public static void main(String[] args) {
        Buffer b = new Buffer();

        Producer p = new Producer(b);
        Consumer c = new Consumer(b);

        p.start();
        c.start();
    }
}
```

week - 7(a)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MouseKeyboardEvents extends JFrame
        implements MouseListener, KeyListener {

    JLabel l;

    MouseKeyboardEvents() {
        l = new JLabel("Click or press any key");
        add(l);

        addMouseListener(this);
        addKeyListener(this);

        setSize(400, 300);
        setTitle("Mouse and Keyboard Events");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
        setFocusable(true);
        requestFocusInWindow();
    }
```

```
    public void mouseClicked(MouseEvent e) {
        l.setText("Mouse Clicked");
    }

    public void mousePressed(MouseEvent e) {
        l.setText("Mouse Pressed");
    }

    public void mouseReleased(MouseEvent e) {
        l.setText("Mouse Released");
    }

    public void mouseEntered(MouseEvent e) {
        l.setText("Mouse Entered");
    }

    public void mouseExited(MouseEvent e) {
        l.setText("Mouse Exited");
    }

    public void keyPressed(KeyEvent e) {
        l.setText("Key Pressed: " + e.getKeyChar());
    }

    public void keyReleased(KeyEvent e) {
        l.setText("Key Released");
    }

    public void keyTyped(KeyEvent e) {
        l.setText("Key Typed: " + e.getKeyChar());
    }

    public static void main(String[] args) {
        new MouseKeyboardEvents();
    }
}

week - 7(b)

import javax.swing.*;
import java.awt.event.*;

class FactorialSwing extends JFrame implements ActionListener {

    JLabel l1, l2;
    JTextField t1, t2;
    JButton b;

    FactorialSwing() {
        l1 = new JLabel("Enter a number:");
        l1.setBounds(30, 30, 120, 30);
```

```
        t1 = new JTextField();
        t1.setBounds(150, 30, 100, 30);

        b = new JButton("Factorial");
        b.setBounds(80, 80, 120, 30);
        b.addActionListener(this);

        l2 = new JLabel("Factorial:");
        l2.setBounds(30, 130, 100, 30);

        t2 = new JTextField();
        t2.setBounds(150, 130, 150, 30);
        t2.setEditable(false);

        add(l1);
        add(t1);
        add(b);
        add(l2);
        add(t2);

        setTitle("Factorial Calculator");
        setSize(350, 230);
        setLayout(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        try {
            int n = Integer.parseInt(t1.getText());

            if (n < 0 || n > 20) {
                t2.setText("Enter 0 to 20");
                return;
            }

            long fact = 1;

            for (int i = 1; i <= n; i++) {
                fact = fact * i;
            }

            t2.setText(String.valueOf(fact));

        } catch (NumberFormatException ex) {
            t2.setText("Invalid input");
        }
    }
```

```
    public static void main(String[] args) {
        new FactorialSwing();
    }
}

week - 8

import javax.swing.*;
import javax.swing.tree.DefaultMutableTreeNode;
import java.awt.*;

class SwingComponents extends JFrame {

    SwingComponents() {
        setTitle("Swing Components Demo");
        setSize(600, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JTabbedPane tabs = new JTabbedPane();

        // JList
        String[] subjects = {
            "Java", "Python", "C", "C++", "DBMS"
        };

        JList<String> list = new JList<>(subjects);
        JScrollPane listScroll = new JScrollPane(list);
        tabs.addTab("JList", listScroll);

        // JTree
        DefaultMutableTreeNode root =
            new DefaultMutableTreeNode("Courses");

        DefaultMutableTreeNode cse =
            new DefaultMutableTreeNode("CSE");
        cse.add(new DefaultMutableTreeNode("Java"));
        cse.add(new DefaultMutableTreeNode("Python"));

        DefaultMutableTreeNode ece =
            new DefaultMutableTreeNode("ECE");
        ece.add(new DefaultMutableTreeNode("Digital Electronics"));
        ece.add(new DefaultMutableTreeNode("Signals"));

        root.add(cse);
        root.add(ece);

        JTree tree = new JTree(root);
        JScrollPane treeScroll = new JScrollPane(tree);
        tabs.addTab("JTree", treeScroll);

        // JTable
```

```
        String[] columns = {"Roll No", "Name", "Marks"};

        Object[][] data = {
            {101, "Ravi", 85},
            {102, "Sita", 90},
            {103, "Arun", 78},
            {104, "Priya", 95}
        };

        JTable table = new JTable(data, columns);
        JScrollPane tableScroll = new JScrollPane(table);
        tabs.addTab("JTable", tableScroll);

        // JScrollPane demonstration
        JTextArea area = new JTextArea(15, 30);
        area.setText(
            "JScrollPane provides scrolling.\n" +
            "It is useful for displaying large content.\n" +
            "Add more text to see the scroll bars.\n".repeat(10)
        );

        JScrollPane textScroll = new JScrollPane(area);
        tabs.addTab("JScrollPane", textScroll);

        add(tabs, BorderLayout.CENTER);

        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new SwingComponents());
    }
}
```

week - 9

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Calculator extends JFrame implements ActionListener {

    JTextField display;
    double num1, num2, result;
    char operator;

    Calculator() {
        setTitle("Simple Calculator");

        display = new JTextField();
        display.setEditable(false);
```

```
        add(display, BorderLayout.NORTH);

        JPanel panel = new JPanel();
        panel.setLayout(new GridLayout(4, 4, 5, 5));

        String buttons[] = {
            "7", "8", "9", "/",
            "4", "5", "6", "*",
            "1", "2", "3", "-",
            "C", "0", "=", "+"
        };

        for (String text : buttons) {
            JButton b = new JButton(text);
            b.addActionListener(this);
            panel.add(b);
        }

        add(panel, BorderLayout.CENTER);

        setSize(300, 350);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        String s = e.getActionCommand();

        try {
            if (s.matches("[0-9]")) {
                display.setText(display.getText() + s);
            } else if (s.equals("C")) {
                display.setText("");
                num1 = num2 = result = 0;
                operator = '\0';
            } else if (s.equals("=")) {
                num2 = Double.parseDouble(display.getText());

                switch (operator) {
                    case '+': result = num1 + num2; break;
                    case '-': result = num1 - num2; break;
                    case '*': result = num1 * num2; break;
                    case '/':
                        if (num2 == 0) {
                            display.setText("Cannot divide by zero");
                            return;
                        }
                        result = num1 / num2;
                        break;
                    default:
```

```
                        return;
                    }

                    display.setText(String.valueOf(result));
                } else {
                    num1 = Double.parseDouble(display.getText());
                    operator = s.charAt(0);
                    display.setText("");
                }
            } catch (NumberFormatException ex) {
                display.setText("Invalid input");
            }
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> new Calculator());
        }
}
```

week - 11